专升本 Java 程序设计模拟卷-卷 9

深度解析讲义

作者：fifine-专升本

日期：2026 年 3 月 2 日

题号	一 (40 分)	二 (20 分)	三 (20 分)	四 (30 分)	五 (40 分)	总分 (150 分)
得分						

一、单项选择题（本大题共 20 小题，每小题 2 分，共 40 分）

1. **Java** 语言是由哪家公司开发的？

   A. Microsoft

   B. Apple

   C. Sun Microsystems

   D. Google

答案与深度解析 答案：**C. Sun Microsystems**

【核心认知框架】

- 题目本质：考查 Java 语言的历史起源与所有权变更
- 关键区分点：初创公司 (Sun) vs 当前维护者 (Oracle) vs 干扰项 (Microsoft/Google)
- 命题人意图：测试基础常识，区分语言起源与当前生态

【选项原理深度拆解】

2. **A. Microsoft** —错误（历史冲突）
   - 历史事实：Microsoft 曾获得 Sun 授权开发 Visual J++，但因私自扩展 API 被 Sun 起诉
   - 结果：Microsoft 转而开发 C# (.NET)，与 Java 竞争
   - 字节码证据：Microsoft JVM 曾支持 `JNative` 等非标准指令，违反 Write Once Run Any
3. **B. Apple** —错误（生态参与者）

fifine专升本

- 历史事实：Apple 是 Java 的早期重要支持者 (Mac OS Runtime Java)
- 现状：OpenJDK 在 macOS 上运行良好，但 Apple 非创造者
- 混淆点：Swift 语言是 Apple 开发的，易混淆

4. **C. Sun Microsystems** —正确（创始者）
   - 历史事实：1995 年由 James Gosling 等在 Sun 公司发布
   - 收购历史：2010 年 Oracle 收购 Sun，Java 归 Oracle 所有
   - 证据：Java 官网版权信息"Copyright © 1995-2023 Oracle and/or its affiliates" (源自 Sun)
5. **D. Google** —错误（生态使用者）
   - 历史事实：Android 早期使用 Java 语法，但 JVM 被 Dalvik/ART 取代
   - 诉讼：Oracle v. Google 案涉及 API 版权，非语言开发权
   - 混淆点：Kotlin 是 Google 推崇的 Android 语言，易混淆

【应背诵知识点】
- 创始公司：Sun Microsystems (1995)
- 当前所有者：Oracle (2010 收购)
- 之父：James Gosling

【命题趋势与实战策略】
- 考场秒杀：看到 Java 起源选 Sun，看到当前维护选 Oracle
- 高频陷阱：混淆 Android (Google) 与 Java 语言本身

6. 下列哪个是 **Java** 中合法的 **boolean** 值？

A. 0

B. false

C. "true"

D. TRUE

答案与深度解析 答案：**B. false**

【核心认知框架】
- 题目本质：考查 Java 基本数据类型 boolean 的字面量规范
- 关键区分点：严格区分大小写、区分布尔值与整数/字符串
- 命题人意图：测试 C/C++ 背景考生的习惯迁移错误

【选项原理深度拆解】

fifine专升本

7. **A. 0** —错误（类型不兼容）
   - **JLS** 规范：JLS §4.12.4 规定 boolean 类型只有 true/false
   - 字节码证据：

```
boolean b = 0; // □□□□□incompatible types: int cannot be converted to boolean
// javap □□□□□□□□□□
```

   - 对比 **C++**：C++ 中 0 可隐式转换为 false，Java 禁止

8. **B. false** —正确（标准字面量）
   - **JLS** 规范：JLS §3.10.3 定义 Boolean 字面量为 `true` 和 `false`
   - 字节码证据：

```
boolean b = false;
// javap -c:
// 0: iconst_0 // boolean □ JVM □□□ int □□□0=false, 1=true
// 1: istore_1
```

   - 内存模型：boolean 数组中每个元素占 1 字节，单个变量可能占 4 字节 (int)

9. **C. "true"** —错误（字符串类型）
   - 类型系统：双引号表示 `java.lang.String` 对象
   - 编译错误：`incompatible types: String cannot be converted to boolean`
   - 转换方法：需使用 `Boolean.parseBoolean("true")`

10. **D. TRUE** —错误（大小写敏感）
    - 语法规则：Java 关键字严格小写
    - 编译错误：`cannot find symbol: variable TRUE`
    - 常量惯例：静态常量常用大写，但布尔字面量必须小写

    【应背诵知识点】
    - boolean 字面量：`true`, `false` (全小写)
    - 不能与 int 互转 (区别于 C)
    - 底层实现：JVM 用 int 表示 boolean (0/1)

    【命题趋势与实战策略】
    - 考场秒杀：看到 boolean 找全小写 true/false
    - 高频陷阱：0/1 混淆，大小写混淆

11. 关于 **switch** 语句，以下说法正确的是？

    A. switch 语句中的表达式可以是 long 类型

fifine专升本

B. case 后面必须跟 break 语句，否则会编译错误

C. switch 语句可以用于 String 类型（Java 7 及以上）

D. default 块必须放在所有 case 之后

答案与深度解析 答案：**C. switch** 语句可以用于 **String** 类型（**Java 7** 及以上）【核心认知框架】：考查 switch 支持的数据类型演变及语法规则。【选项深度拆解】：

- **A. long** 类型：错误。switch 支持 byte, short, char, int, enum, String, 包装类。long 超出 tableswitch/lookupswitch 索引范围。
- **B.** 必须 **break**：错误。break 可选，缺失会导致穿透 (fall-through)，编译通过。
- **C. String** 类型：正确。Java 7 引入，底层通过 `hashCode()` 和 `equals()` 实现。
- **D. default** 位置：错误。default 可放在任意位置，不影响逻辑。

【字节码证据】：String switch 编译后会生成两个 switch，第一个查 hashCode，第二个查 equals。

12. 下列哪个循环结构至少会执行一次循环体？

答案与深度解析 答案：**C. do-while** 循环【核心认知框架】：考查循环执行流程差异。【选项深度拆解】：

- **for/while**：先判断后执行，可能 0 次。
- **do-while**：先执行后判断，至少 1 次。
- 字节码：`do-while` 无条件跳转指令在循环体后。

13. 关于数组的定义，正确的是？

答案与深度解析 答案：**C. int[] arr = {1,2,3,4,5};**【核心认知框架】：考查数组初始化语法。【选项深度拆解】：

- **A/B**：错误。new int[5] 后不能跟初始化列表。
- **C**：正确。静态初始化省略 new 和类型。
- **D**：正确语法但 C 更标准，D 在类成员变量合法，局部变量推荐 C 风格。题目通常选 C 为最佳实践。
- 字节码：`anewarray` 指令创建数组，`astore` 存储。

14. 下列哪个选项可以正确定义一个有 **3** 行 **4** 列的二维数组？

答案与深度解析 答案：**A. int[][] arr = new int[3][4];**【核心认知框架】：考查多维数组内存结构。【选项深度拆解】：

- **A**：正确。Java 二维数组是数组的数组。

fifine专升本

- **B/C/D**：语法错误。定义时不能指定第二维大小而不指定第一维 (除非初始化)。
- 内存模型：3 个引用指向 3 个长度为 4 的 int 数组对象。

15. 在 **Java** 中，所有类的直接或间接父类是？

答案与深度解析 答案：**A. Object**【核心认知框架】：考查类层次结构根节点。【选项深度拆解】：
- **Object**：正确。JLS 规定所有类继承 Object。
- **Class**：错误。Class 是类的元数据类。
- 字节码：`javap -c` 显示所有类默认 `extends java/lang/Object`。

16. 下列关于 **this** 关键字的说法，正确的是？

答案与深度解析 答案：**B. this** 可以调用本类的其他构造方法【核心认知框架】：考查 this 关键字用途。【选项深度拆解】：
- **A**：错误。静态方法无 this 上下文。
- **B**：正确。`this(...)` 必须在构造器首行。
- **C**：错误。this 指向当前对象，super 指向父类。
- **D**：错误。this 可作为返回值返回当前对象引用。

17. 关于方法重写（**Override**）的规则，错误的是？

答案与深度解析 答案：**D.** 父类的私有方法可以被子类重写【核心认知框架】：考查多态与访问控制。【选项深度拆解】：
- **D**：错误。private 方法不可见，子类定义同名方法是隐藏而非重写。
- **A/B/C**：正确。遵循里氏替换原则。
- 字节码：`invokevirtual` 动态绑定，private 方法用 `invokespecial`。

18. 下列哪个修饰符可以修饰局部变量？

答案与深度解析 答案：**D. final**【核心认知框架】：考查修饰符作用域。【选项深度拆解】：
- **public/private/protected**：错误。仅修饰成员。
- **final**：正确。表示引用不可变。
- 字节码：局部变量表中标记 ACC_FINAL。

19. 关于接口，以下说法错误的是？

fifine专升本

答案与深度解析 答案：**D.** 接口可以包含实例变量 【核心认知框架】：考查接口成员规范。【选项深度拆解】：

- **D**：错误。接口字段隐含 public static final，无实例变量。
- **A/B/C**：正确。Java 8/9 新特性。
- **JLS**：§9.3 接口成员必须是静态常量。

20. 下列哪个类是所有异常类的父类？

答案与深度解析 答案：**C. Throwable** 【核心认知框架】：考查异常层次结构。【选项深度拆解】：

- **Throwable**：正确。Exception 和 Error 继承自它。
- **Exception**：错误。Throwable 的子类。
- 字节码：`athrow` 指令抛出 Throwable 对象。

21. 在异常处理中，**finally** 块会在什么时候执行？

答案与深度解析 答案：**C.** 无论是否发生异常都会执行（除非 **JVM** 退出）【核心认知框架】：考查异常控制流。【选项深度拆解】：

- **C**：正确。JSR/RET 指令或新实现保证执行。
- 例外：`System.exit(0)` 或断电。
- 字节码：异常表 (Exception Table) 包含 finally 范围。

22. 下列哪个集合类是基于哈希表实现的，并且不允许存储重复元素？

答案与深度解析 答案：**C. HashSet** 【核心认知框架】：考查集合底层实现。【选项深度拆解】：

- **HashSet**：正确。底层 HashMap，Key 不可重复。
- **ArrayList/LinkedList**：错误。列表允许重复。
- **HashMap**：错误。是 Map 不是 Collection。
- 哈希机制：`hashCode()` + `equals()` 判重。

23. 关于泛型，以下说法正确的是？

答案与深度解析 答案：**C.** 泛型可以用于类、接口、方法 【核心认知框架】：考查泛型适用范围。【选项深度拆解】：

- **A**：错误。泛型参数必须是引用类型。
- **B**：错误。类型擦除，运行时不可获取。
- **C**：正确。全面支持。

ffine专升本

- **D**：错误。支持通配符?。
- 字节码：Signature 属性保留泛型信息，但运行时擦除为 Object。

24. 在 **Java** 中，实现 **Runnable** 接口与继承 **Thread** 类相比，优势是？

答案与深度解析 答案：**D.** 以上都是【核心认知框架】：考查多线程实现方式。【选项深度拆解】：
- **A**：正确。避免单继承限制。
- **B**：正确。资源共享更灵活。
- **C**：正确。解耦任务与线程。
- 设计模式：策略模式体现。

25. 关于 **synchronized** 关键字，以下说法正确的是？

答案与深度解析 答案：**C.** 可以修饰静态方法，锁定的是当前类的 **Class** 对象【核心认知框架】：考查锁机制。【选项深度拆解】：
- **A**：错误。不能修饰类。
- **B**：错误。不能修饰构造方法。
- **C**：正确。静态锁锁 Class 对象。
- **D**：错误。不能修饰变量。
- 字节码：`ACC_SYNCHRONIZED` 标志或 `monitorenter/monitorexit`。

26. 在 **Java I/O** 中，**File** 类的主要作用是？

答案与深度解析 答案：**B.** 管理文件和目录的元信息（如创建、删除、重命名）【核心认知框架】：考查 File 类职责。【选项深度拆解】：
- **B**：正确。File 是抽象路径名，不读写内容。
- **A**：错误。IO 流负责读写。
- **C**：错误。Buffered 流负责缓冲。
- **D**：错误。Serializable 接口负责序列化。

27. 下列哪个类用于在 **JDBC** 中建立数据库连接？

答案与深度解析 答案：**C. DriverManager** 【核心认知框架】：考查 JDBC 核心类。【选项深度拆解】：
- **DriverManager**：正确。`getConnection()` 获取 Connection。
- **Connection**：错误。是连接对象，非建立者。
- **Statement**：错误。执行 SQL。

fine专升本

- **ResultSet**：错误。结果集。

28. 关于 **Swing** 中 **JButton** 的事件处理，以下说法正确的是？

答案与深度解析 答案：**B.** 需要实现 **ActionListener** 接口并注册到按钮 【核心认知框架】：考查事件监听机制。【选项深度拆解】：
- **B**：正确。标准观察者模式。
- **A**：错误。触发 ActionEvent。
- **C**：错误。可添加多个监听器。
- **D**：错误。默认 FlowLayout。

29. 在 **Java** 中，下列哪个关键字用于引入包中的类？

答案与深度解析 答案：**B. import** 【核心认知框架】：考查包管理语法。【选项深度拆解】：
- **import**：正确。编译期指令。
- **package**：错误。定义包。
- **include/using**：错误。C/C++/C# 语法。
- 字节码：常量池中存储类全限定名，import 不影响字节码。

ffine专升本

二、判断题（本大题共 10 小题，每小题 2 分，共 20 分）

1. **Java** 程序源文件名必须与 **public** 类名相同，且扩展名为**.java**。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查编译单元规范。【深度解析】：
- **JLS** 规范：§7.6 规定 public 类必须与文件名一致。
- 编译器行为：`javac` 强制检查，否则报错。
- 例外：非 public 类文件名可任意，但惯例保持一致。
- 字节码：生成的 `.class` 文件名与类名一致，与源文件名无关 (除 public 类)。

2. 在同一个类中，可以定义两个同名的方法，只要它们的参数列表不同即可。（　）

答案与深度解析 答案：√ 【核心认知框架】：考查方法重载 (Overloading)。【深度解析】：
- 定义：方法名相同，参数类型/个数/顺序不同。
- 字节码：方法描述符 (Descriptor) 不同，如 `(I)V` vs `(Ljava/lang/String;)V`。
- 返回类型：返回类型不同不足以区分重载。

3. 子类不能继承父类的私有成员。（　）

答案与深度解析 答案：√【核心认知框架】：考查访问控制与继承。【深度解析】：
- 继承性：私有成员在子类内存中存在，但不可直接访问。
- 语义：通常说”不可继承”指不可见。
- 访问：可通过父类 public/protected 方法间接访问。
- 字节码：子类字节码中无父类 private 字段/方法的直接引用权限。

4. 接口中的成员变量默认是 **public static final** 的。（　）

答案与深度解析 答案：√【核心认知框架】：考查接口字段隐含修饰符。【深度解析】：
- **JLS**：§9.3 强制规定。
- 编译器：自动补全修饰符。
- 设计意图：接口用于定义常量契约，非状态存储。

5. **throw** 和 **throws** 关键字的作用相同，都是用来抛出异常。（　）

答案与深度解析 答案：×【核心认知框架】：考查异常关键字区别。【深度解析】：
- **throw**：动词，用于方法体内抛出异常对象。
- **throws**：声明，用于方法签名声明可能抛出的异常类型。
- 字节码：`athrow` 指令对应 throw，异常表对应 throws。

6. **ArrayList** 和 **LinkedList** 都是线程安全的集合类。（　）

答案与深度解析 答案：×【核心认知框架】：考查集合线程安全性。【深度解析】：
- 事实：两者均非线程安全。
- 并发修改：多线程操作可能抛 `ConcurrentModificationException` 或数据不一致。
- 解决方案：使用 `Collections.synchronizedList` 或 `CopyOnWriteArrayList`。

7. 泛型方法可以在非泛型类中定义。（　）

答案与深度解析 答案：√【核心认知框架】：考查泛型方法独立性。【深度解析】：
- 语法：`<T> void method(T t)` 可定义在普通类中。
- 作用域：泛型参数仅在该方法内有效。
- 字节码：方法属性 Signature 包含泛型信息。

8. 调用 **Thread.sleep()** 方法会使当前线程暂停执行，但不会释放对象锁。（　）

fifine专升本

答案与深度解析 答案：√【核心认知框架】：考查线程状态与锁。【深度解析】：
- 行为：sleep 进入 TIMED_WAITING 状态，不释放 monitor。
- 对比：`wait()` 会释放锁。
- 场景：适用于需持有锁暂停的场景 (少见)。

9. **FileReader** 和 **FileWriter** 是字节流类。（　）

答案与深度解析 答案：×【核心认知框架】：考查 IO 流分类。【深度解析】：
- 事实：它们是字符流 (Reader/Writer)。
- 字节流：`FileInputStream/FileOutputStream`。
- 编码：字符流涉及 charset 编码转换。

10. 在 **UDP** 通信中，**DatagramSocket** 负责发送和接收数据报，**DatagramPacket** 负责封装数据。（　）

答案与深度解析 答案：√【核心认知框架】：考查网络编程 API。【深度解析】：
- **Socket**：端点，负责传输。
- **Packet**：数据包，承载数据、IP、端口。
- 协议：UDP 无连接，包独立。

三、填空题（本大题共 5 小题，每空 2 分，共 20 分）

1. **Java** 的数据类型分为两大类：_____ 类型和 _____ 类型。

答案与深度解析 答案：基本 **(Primitive)**、引用 **(Reference)**【核心认知框架】：考查类型系统分类。【深度解析】：
- 基本类型：8 种 (byte, short, int, long, float, double, char, boolean)，栈上分配值。
- 引用类型：类、接口、数组，堆上分配对象，栈存引用。
- 字节码：基本类型直接操作值，引用类型操作对象地址。

2. 面向对象程序设计的三个特征是封装、_____ 和 _____。

答案与深度解析 答案：继承、多态【核心认知框架】：考查 OOP 核心概念。【深度解析】：
- 封装：隐藏内部实现，暴露接口。
- 继承：代码复用，is-a 关系。
- 多态：同一接口不同实现，动态绑定。
- 字节码：多态体现为 `invokevirtual` 动态分派。

ffine专升本

3. 在 **Java** 中，使用关键字 _____ 定义常量，使用关键字 _____ 定义抽象方法。

> 答案与深度解析 答案：**final**、**abstract** 【核心认知框架】：考查修饰词语义。【深度解析】：
> - **final**：不可变性，编译期常量或运行期常量。
> - **abstract**：无实现，强制子类重写。
> - 互斥：final 方法不能被 abstract 修饰。

4. 线程的优先级可以通过 _____ 方法设置，线程休眠可以使用 _____ 方法。

> 答案与深度解析 答案：**setPriority()**、**sleep()** 【核心认知框架】：考查 Thread API。【深度解析】：
> - 优先级：1-10，依赖操作系统调度，不保证执行顺序。
> - 休眠：静态方法，作用于当前线程，不释放锁。
> - 异常：sleep 需处理 InterruptedException。

5. 集合框架中，**Map** 接口的常用实现类包括 _____ 和 _____。

> 答案与深度解析 答案：**HashMap**、**TreeMap** (或 **Hashtable**) 【核心认知框架】：考查集合体系。【深度解析】：
> - **HashMap**：哈希表，无序，允许 null。
> - **TreeMap**：红黑树，有序，不允许 null key。
> - **Hashtable**：遗留类，线程安全，不建议使用。

fifine专升本

四、程序运行结果题（本大题共 5 小题，每小题 6 分，共 30 分）

1. 写出下列程序的输出结果：

```
public class Test1 {
public static void main(String[] args) {
int sum = 0;
for (int i = 1; i <= 5; i++) {
switch (i) {
case 1:
case 2: sum += i; break;
case 3: sum += i * 2; break;
default: sum += i;
```

```
}
}
System.out.println(sum);
}
}
```

答案与深度解析 答案：**14**【核心认知框架】：考查 switch 穿透性与循环累加。【执行流程分析】：

- **i=1**：case 1 穿透到 case 2，sum += 1 (实际 case 1 无代码，执行 case 2 代码 sum+=1)，break。sum=1。
- **i=2**：case 2，sum += 2，break。sum=3。
- **i=3**：case 3，sum += 3*2=6，break。sum=9。
- **i=4**：default，sum += 4，无 break 但循环结束。sum=13。
- **i=5**：default，sum += 5。sum=18。
- 修正：case 1 无代码，直接落入 case 2。i=1 时 sum+=1。i=2 时 sum+=2。i=3 时 sum+=6。i=4,5 时 sum+=4+5=9。总计 1+2+6+9 = 18。
- 再次检查：case 1 后无 break，落入 case 2。case 2 有 break。
- i=1: 进入 case 1 -> 落入 case 2 -> sum+=1 -> break. (sum=1)
- i=2: 进入 case 2 -> sum+=2 -> break. (sum=3)
- i=3: 进入 case 3 -> sum+=6 -> break. (sum=9)
- i=4: 进入 default -> sum+=4. (sum=13)
- i=5: 进入 default -> sum+=5. (sum=18)
- 最终结果：18。
- 注意：若题目意图 case 1 不执行加法，则 sum=17。但语法上 case 1 落入 case 2。
- 标准答案修正：通常此类题 case 1 无代码意为落入。结果 18。

【字节码证据】：`tableswitch` 指令实现跳转，fall-through 由字节码顺序决定。

fifine专升本

2. 写出下列程序的输出结果：

```java
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { System.out.print("C "); } }
public class Test2 {
public static void main(String[] args) {
C c = new C();
}
}
```

答案与深度解析 答案：**A B C** 【核心认知框架】：考查构造器调用链。【执行流程分析】：

- 初始化顺序：父类 -> 子类。
- 隐式调用：子类构造器首行隐式含 `super()`。
- 流程：new C() -> C 构造 -> B 构造 -> A 构造 -> A 执行 -> B 执行 -> C 执行。
- 字节码：`invokespecial` 调用父类构造器 `<init>`。

3. 写出下列程序的输出结果：

```java
public class Test3 {
public static void main(String[] args) {
Integer a = 127;
Integer b = 127;
Integer c = 128;
Integer d = 128;
System.out.println(a == b);
System.out.println(c == d);
}
}
```

答案与深度解析 答案：**true false** 【核心认知框架】：考查整数缓存池 (Integer Cache)。【执行流程分析】：

- 自动装箱：`Integer.valueOf(int)`。
- 缓存范围：-128 到 127 复用对象。
- **a==b**：127 在缓存内，引用相同，true。
- **c==d**：128 超出缓存，new 对象，引用不同，false。
- 字节码:`invokestatic #java/lang/Integer.valueOf(I)Ljava/lang/Integer;`。

fine专升本

4. 写出下列程序的输出结果：

```java
public class Test4 {
public static void main(String[] args) {
int[] arr = {1, 2, 3, 4, 5};
try {
System.out.println(arr[5]);
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("????");
} catch (Exception e) {
System.out.println("????");
} finally {
```

```
System.out.println("finally □");
}
System.out.println("□□□□");
}
}
```

答案与深度解析 答案：

```
□□□□
finally □
□□□□
```

【核心认知框架】：考查异常捕获顺序与 finally 执行。【执行流程分析】：

- 异常抛出：`arr[5]` 抛 `ArrayIndexOutOfBoundsException`。
- 捕获匹配：第一个 catch 精确匹配，执行。
- **finally**：无论是否异常，必执行。
- 后续代码：异常被捕获，程序继续执行。
- 字节码：异常表 (Exception Table) 定义捕获范围。

5. 写出下列程序的输出结果：

```
class Counter {
static int count = 0;
public Counter() { count++; }
public static int getCount() { return count; }
}
public class Test5 {
public static void main(String[] args) {
Counter c1 = new Counter();
Counter c2 = new Counter();
Counter c3 = new Counter();
System.out.println(Counter.getCount());
}
}
```

答案与深度解析 答案：**3**【核心认知框架】：考查 static 变量共享性。【执行流程分析】：

- 静态变量：属于类，所有实例共享。
- 构造器：每次 new 调用，count 累加。

fifine专升本

- 结果：new 3 次，count=3。
- 内存：count 存储在方法区 (静态区)。

五、编程题（本大题共 2 小题，每小题 20 分，共 40 分）

1. 设计一个”动物”类层次结构

答案与深度解析 答案：见代码实现 【核心认知框架】：考查抽象类、继承、多态、集合遍历。【设计思路深度解析】：

- 抽象类 **Animal**：定义通用属性 name, age 和抽象方法 makeSound。
- 子类 **Dog/Cat**：扩展特有属性，实现抽象方法。
- 多态：`ArrayList<Animal>` 存储子类对象。
- 类型判断：使用 `instanceof` 进行向下转型。
- 最佳实践：遵循开闭原则，易于扩展新动物。

【参考代码】：

```java
abstract class Animal {
    protected String name;
    protected int age;
    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public abstract void makeSound();
    public String getName() { return name; }
    public int getAge() { return age; }
}

class Dog extends Animal {
    private String breed;
    public Dog(String name, int age, String breed) {
        super(name, age);
        this.breed = breed;
    }
    public void makeSound() { System.out.println("汪汪"); }
    public String getBreed() { return breed; }
}

class Cat extends Animal {
```

fifine专升本

```java
    private String color;
    public Cat(String name, int age, String color) {
        super(name, age);
        this.color = color;
    }
    public void makeSound() { System.out.println("□□"); }
    public String getColor() { return color; }
}

public class Main {
    public static void main(String[] args) {
        ArrayList<Animal> list = new ArrayList<>();
        list.add(new Dog("□□", 3, "□□□"));
        list.add(new Cat("□□", 2, "□□"));
        for (Animal a : list) {
            a.makeSound();
            System.out.println(a.getName() + "," + a.getAge());
            if (a instanceof Dog) {
                System.out.println("□□:" + ((Dog)a).getBreed());
            } else if (a instanceof Cat) {
                System.out.println("□□:" + ((Cat)a).getColor());
            }
        }
    }
}
```

fifine专升本

【考点延伸】:

- **instanceof**：运行时类型检查，避免 ClassCastException。
- 泛型：`ArrayList<Animal>` 保证类型安全。
- 多态：调用 makeSound 时动态绑定子类实现。

2. 实现一个简单的”图书馆”管理系统

答案与深度解析 答案：见代码实现 【核心认知框架】: 考查 Map 集合应用、业务逻辑封装、异常处理。【设计思路深度解析】:

- 数据结构：`HashMap<String, Book>` 利用 ID 快速查找 (O(1))。
- 封装：Book 类私有属性，Library 类管理业务逻辑。
- 状态管理：isBorrowed 标志位控制借还逻辑。

- 健壮性：检查 null 和状态，避免逻辑错误。

【参考代码】:

```java
class Book {
    private String id, title, author;
    private boolean isBorrowed;
    // ???getter/setter/toString ??
    public Book(String id, String title, String author) {
        this.id = id; this.title = title; this.author = author;
    }
    public String getId() { return id; }
    public boolean isBorrowed() { return isBorrowed; }
    public void setBorrowed(boolean b) { isBorrowed = b; }
    public String toString() { return id + ":" + title + ":" + author
        ; }
}

class Library {
    private HashMap<String, Book> books = new HashMap<>();
    public void addBook(Book book) {
        if (books.containsKey(book.getId())) {
            System.out.println("??????");
        } else {
            books.put(book.getId(), book);
        }
    }
    public void borrowBook(String id) {
        Book b = books.get(id);
        if (b == null) { System.out.println("?????"); }
        else if (b.isBorrowed()) { System.out.println("???"); }
        else { b.setBorrowed(true); System.out.println("?????"); }
    }
    // removeBook, returnBook, searchBook, displayAllBooks ????
}
```

【考点延伸】:

- **HashMap**：key 唯一性用于管理图书 ID。
- 业务逻辑：借书前检查状态，防止重复借出。
- 扩展性：可轻松添加预约、罚款等功能。

fifine专升本

——试卷结束，祝考试顺利——

fifine专升本